Anomaly Detection in Encrypted Network Traffic Using Machine Learning

A PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

BY

G. Tanuja — **22331A4422**

B. Chandini — **22331A4404**

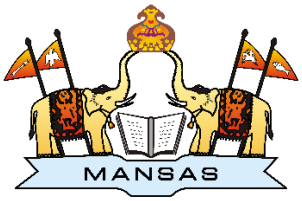
S. Sai kiran — **22331A4444**

D. Sagar — **23335A4403**

Under the Supervision of

Dr. P. Srinivasa Rao

Professor



DEPARTMENT OF DATA ENGINEERING

MAHARAJ VIJAYARAM GAJAPATHI RAJ COLLEGE OF ENGINEERING

(Autonomous)

(Approved by AICTE, New Delhi, and permanently affiliated to JNTUGV, Vizianagaram), Listed u/s 2(f) &

12(B) of UGC Act 1956.

Vijayaram Nagar Campus, Chintalavalasa, Vizianagaram-535005, Andhra Pradesh APRIL,

2026

CERTIFICATE

This is to certify that the project report entitled "**Anomaly Detection in Encrypted Network Traffic Using Machine Learning**" being submitted by G.Tanuja(22331A4422), B.Chandini(22331A4404), S.Sai Kiran(22331A4444), D.Sagar(23335A4403) in partial fulfillment for the award of the degree of "**Bachelor of Technology" in Computer Science and Engineering (Data Science)** is a record of bonafide work done by them under my supervision during the academic year 2025-2026.

Dr. P. Srinivasa Rao
Professor,
Supervisor,
Department of Data Engineering,
MVGR College of Engineering(A),
Vizianagaram.

Dr. V. Jyothi
Associate Professor,
Head of the Department,
Department of Data Engineering,
MVGR College of Engineering(A),
Vizianagaram.

External Examiner

DECLARATION

We hereby declare that the work done on the dissertation entitled "**Anomaly Detection in Encrypted Network Traffic Using Machine Learning**" has been carried out by us and submitted in partial fulfilment for the award of credits in Bachelor of Technology in Computer Science and Engineering- Data Science, MVGR College of Engineering (Autonomous) and affiliated to JNTUGV, Vizianagaram. The various contents incorporated in the dissertation have not been submitted for the award of any degree of any other institution or university.

ACKNOWLEDGEMENTS

.We express our sincere gratitude to **Dr. P. Srinivasa Rao** for his invaluable guidance and support as our mentor throughout the project. His unwavering commitment to excellence and constructive feedback motivated us to achieve our project goals. We are greatly indebted to his for his exceptional guidance. We would like to express our heartfelt thanks to our Project Coordinator **Dr. P. Srinivasa Rao** for the constant support, and motivation provided during the project. Your direction played a key role in completing our work successfully

Additionally, we extend our thanks to **Prof. P.S. Sitharama Raju (Director)**, **Prof. Y.M.C. Shekar (Principal)**, and **Dr. V. Jyothi (Head of the Department)** for their unwavering support and assistance, which were instrumental in the successful completion of the project. We also acknowledge the dedicated assistance provided by all the staff members in the Department of Data Engineering. Finally, we appreciate the contributions of all those who directly or indirectly contributed to the successful execution of this endeavour.

G. Tanuja (**22331A4422)**
B. Chandini (**22331A4404)**
S. Sai Kiran (**22331A4444)**
D. Sagar(**23335A4403**)

ABSTRACT

This project implements a robust Hybrid Intrusion Detection System (HIDS) that leverages a multi-stage machine learning pipeline to secure network environments against sophisticated cyberattacks. The architecture begins with a comprehensive Data Preprocessing phase, where raw network flow logs are cleaned and transformed via a StandardScaler to ensure that varied features, such as packet counts and byte rates, maintain uniform influence during analysis. At its core, the system employs a unique two-tier detection logic: a Deep Learning Autoencoder is first trained to model the "latent space" of normal traffic, while an Isolation Forest acts as a secondary filter to scrutinize any data points that return a high Reconstruction Error. This hybrid approach allows the model to differentiate between benign network fluctuations and actual malicious intent with high precision. To make these findings actionable, the system is integrated into a Flask-based web dashboard that provides real-time visibility into the network's health. By mapping detected anomalies to specific threat categories like Bruteforce, DDoS, or CryptoMiner, and visualizing the results through interactive charts, the system empowers security administrators to move from passive monitoring to proactive threat hunting.

CONTENTS

List of Abbreviations

- AE – Autoencoder
- API – Application Programming Interface
- DL – Deep Learning
- DPI – Deep Packet Inspection
- HTTPS – Hypertext Transfer Protocol Secure
- IDS – Intrusion Detection System
- IF – Isolation Forest
- ML – Machine Learning
- NumPy – Numerical Python Library
- Pandas – Python Data Analysis Library
- ReLU – Rectified Linear Unit
- TCP – Transmission Control Protocol
- TLS – Transport Layer Security
- UDP – User Datagram Protocol
- UI – User Interface
- VPN – Virtual Private Network

List of Figures

List of Tables

CHAPTER 1

INTRODUCTION

The rapid expansion of digital communication networks has led to an unprecedented increase in encrypted internet traffic, with modern infrastructures relying heavily on protocols such as HTTPS, TLS, and VPN to ensure confidentiality and data integrity. Encryption has shifted from a specialized tool to a fundamental requirement across critical sectors, including finance, healthcare, and e-commerce. Recent global traffic analyses indicate that a dominant portion of all internet communication is now encrypted by default, reflecting a collective move toward a more secure and private digital landscape. While this shift significantly enhances user privacy and data protection, it simultaneously obscures the visibility required for traditional network monitoring, creating a complex environment where security and privacy are often in direct tension.

This lack of visibility presents a critical challenge for existing network security systems, as traditional Intrusion Detection Systems (IDS) were primarily designed to inspect unencrypted packet payloads. In an unencrypted environment, security engines use Deep Packet Inspection (DPI) to identify malicious patterns, signatures, and unauthorized data transfers. However, when traffic is encrypted, the payload is transformed into ciphertext, rendering these inspection engines ineffective. This shift has inadvertently provided a "cloak" for cyber adversaries, who now exploit encrypted tunnels to bypass perimeter defenses, mask malware delivery, and maintain command-and-control (C2) communications. Consequently, there is an urgent need for advanced detection strategies that can identify threats based on behavioral metadata rather than the inaccessible contents of the packets themselves.

1.1 Problem Statement

Traditional Intrusion Detection Systems (IDS) and firewall mechanisms are designed to inspect packet payloads using signature-based or rule-based detection strategies. These systems depend on Deep Packet Inspection (DPI) to examine the contents of packets for malicious patterns. However, in encrypted traffic, the payload is transformed into ciphertext, making it inaccessible to inspection engines. This limitation has created an "encryption blind spot" where attackers exploit secure tunnels to mask malware delivery, command-and-control (C2) channels, and data exfiltration. While decryption at gateways is a common countermeasure, it

introduces significant computational latency, weakens end-to-end security guarantees, and raises serious regulatory and privacy compliance concerns.

1.2 Objective

The primary objective of this research is to develop a scalable, protocol-independent, and privacy-preserving anomaly detection system capable of identifying threats without traffic decryption. By shifting the focus from payload inspection to behavioral metadata analysis—such as flow duration, packet counts, and inter-arrival times—this project aims to distinguish between benign and malicious activities. The ultimate goal is to provide a robust security framework that maintains high detection accuracy while fully respecting the integrity of encrypted communications.

1.3 Existing Models

Since encrypted traffic obscures the internal data of a packet, researchers have shifted away from content-based detection toward metadata-driven analysis. Current literature focuses on extracting patterns from flow characteristics to identify anomalies without the need for decryption.

1.3.1 Flow-Based Feature Analysis

This method involves analyzing high-level traffic statistics such as packet counts, byte volumes, and flow duration. Ibraheem et al. (2022) demonstrated that utilizing flow-based features in conjunction with Support Vector Machines (SVM) and Random Forest models allows for effective anomaly detection in HTTPS traffic. By focusing on how data moves rather than what it contains, this approach bypasses the limitations of encryption.

1.3.2 Deep Learning Reconstruction

Advanced unsupervised models, specifically Autoencoders, are used to learn the "normal" state of encrypted communications. Monroy et al. (2023) applied this technique to DNS-over-HTTPS (DoH) traffic. By training the model only on benign data, any malicious activity results in a high reconstruction error, signaling an anomaly. This is particularly effective for identifying sophisticated tunneling attempts within encrypted streams.

1.3.3 Algorithmic Classification

Broader research, such as the study conducted by Ji et al. (2024), has synthesized the effectiveness of various machine learning algorithms across general encrypted traffic types.

Their findings identified Random Forest as a consistently high performer due to its ability to handle non-linear relationships in traffic metadata. However, much of this research highlights the need for more direct experimental validation in real-time environments.

1.3.4 Behavioural Metadata Detection

General research in the field of Anomaly-based Intrusion Detection Systems (IDS) focuses on behavioural deviations. Instead of looking for specific attack signatures, these systems monitor for shifts in communication metadata that deviate from an established baseline. While excellent for detecting unknown threats, these systems require high-quality metadata to minimize false alarm rates.

Method	**Detection Attributes**	**Example Models / Studies**
Flow-Based Analysis	Flow duration, packet counts, byte volume	SVM, Random Forest
Reconstruction Analysis	Reconstruction error, latent feature deviations	Autoencoders
Algorithmic Classification	Non-linear metadata correlations	Random Forest
Behavioral Detection	Protocol patterns, behavioral metadata	Anomaly-based IDS

Table 1.3.4: Detection methods

1.4 Proposed Model

To address the shortcomings of existing methodologies, this work proposes a hybrid anomaly detection framework. The system integrates two complementary unsupervised models: a Deep Learning Autoencoder and an Isolation Forest algorithm. The Autoencoder serves as a sophisticated feature extractor, learning the compressed, latent representations of "normal" traffic behaviour. Traffic that deviates from this learned baseline results in a high reconstruction error, flagging it as a potential threat.

The compressed features are then processed by the Isolation Forest, which detects anomalies by isolating rare instances in the feature space through random partitioning. This dual-layered approach creates a decision fusion strategy that enhances robustness; the Autoencoder handles the complexity of high-dimensional metadata, while the Isolation Forest provides a statistical check that reduces false positives. This framework is evaluated against benchmark datasets to demonstrate that malicious behaviour can be identified solely through metadata characteristics.

CHAPTER 2

LITERATURE SURVEY

The increasing dominance of encrypted network traffic has motivated extensive research into alternative intrusion detection mechanisms that do not rely on payload inspection. Researchers have explored flow-based analysis, statistical modeling, and machine learning techniques to detect anomalies in encrypted environments. This section reviews significant contributions in encrypted traffic analysis and highlights the limitations addressed in the present work.

[1] Lotfollahi et al., 2020, in the paper "Deep Packet: A Novel Approach for Encrypted Traffic Classification Using Deep Learning," proposed a deep learning-based framework that uses neural networks to automatically learn patterns from packet headers and statistical flow features. Their work demonstrated that encrypted traffic can still be classified accurately using metadata such as packet length, inter-arrival time, and flow duration without accessing the encrypted content. [2] Shapira et al., 2021, in *"*Flow-Based Machine Learning Techniques for Encrypted Traffic Classification," investigated the use of machine learning algorithms trained on flow-based attributes like packet size distribution, timing characteristics, and transport-layer information to identify application types within encrypted traffic. Their results showed that these features provide strong discriminative power even when payload data is unavailable. [3] Zhang et al., 2023, in *"*A Survey on Encrypted Traffic Classification Using Machine Learning and Deep Learning," conducted a comprehensive review of various encrypted traffic analysis techniques and highlighted the shift from traditional statistical methods to deep learning models such as convolutional neural networks and recurrent neural networks. The survey emphasized that deep learning models are capable of automatically extracting relevant features from raw traffic data, but many existing approaches require large labeled datasets and may not perform well when detecting previously unseen attacks. [4] Monroy et al., 2023, in "Autoencoder-Based Detection of Malicious DNS-over-HTTPS Traffic," introduced a reconstruction-based anomaly detection method using Autoencoders. In this approach, the model learns the normal patterns of encrypted traffic during training and calculates reconstruction error during testing; unusually high errors indicate potential malicious activity. Their study demonstrated that Autoencoder models are effective for detecting abnormal behavior in encrypted communication channels such as DNS-over-HTTPS. [5] Wang et al., 2022, in "Machine Learning-Based Flow-Level Anomaly Detection in Network Traffic,*"* evaluated several machine learning models for detecting abnormal network flows and found that the Isolation

Forest algorithm performs well for identifying sparse anomalies in high-dimensional datasets by isolating unusual observations through random partitioning of the feature space. The study highlighted the efficiency and scalability of tree-based approaches for large network datasets. [6] De Albuquerque Filho et al., 2022, in "Neural Network-Based Hybrid Models for Network Anomaly Detection," analyzed different hybrid detection architectures that combine neural networks with other machine learning algorithms. Their review concluded that hybrid systems generally provide better detection accuracy and lower false-positive rates because different algorithms complement each other's strengths and weaknesses. [7] Xu et al., 2025, in "Unsupervised Deep Learning Approaches for Intrusion Detection Systems*,"* discussed the importance of unsupervised learning models for modern cybersecurity applications. The authors highlighted that Autoencoders and similar deep learning techniques are particularly useful for detecting zero-day attacks because they learn representations of normal network behavior and identify deviations without relying on predefined attack signatures. Despite these advancements, several challenges remain, including selecting appropriate anomaly thresholds, handling high-dimensional traffic data, and ensuring that models can generalize to new types of attacks. These studies collectively demonstrate that combining deep learning techniques such as Autoencoders with machine learning algorithms like Isolation Forest can significantly improve anomaly detection performance in encrypted network environments while maintaining user privacy and scalability.

CHAPTER 3

THEORITICAL BACKGROUND

3.1 Machine learning

3.1.1 What is Machine Learning?

Machine learning is an application of AI that enables systems to learn and improve from experience without being explicitly programmed. Machine learning focuses on developing computer programs that can access data and use it to learn for themselves. Machine learning is something that is capable to imitate the intelligence of the human behavior. Machine learning is used to perform complex tasks in a way that humans solve the problems. Machine learning can be descriptive it uses the data to explain, predictive, and prescription.

3.1.2 Why Machine Learning?

Machine learning involves computers learning from data provided so that they carry out certain tasks. For more advanced tasks, it can be challenging for a human to manually create the needed algorithms. In practice, it can turn out to be more effective to help the machine develop its own algorithm, rather than having human programmers specify every needed step. The discipline of machine learning employs various approaches to teach computers to accomplish tasks where no fully satisfactory algorithm is available. In cases where vast numbers of potential answers exist, one approach is to label some of the correct answers as valid. This can then be used as training data for the computer to improve the algorithms it uses to determine correct answers. The nearly limitless quantity of available data, affordable data storage, and growth of less expensive and more powerful processing has propelled the growth of ML. Now many industries are developing more robust models capable of analysing bigger and more complex data while delivering faster, more accurate results on vast scales. ML tools enable organizations to more quickly identify profitable opportunities and potential risks. The practical applications of machine learning drive business results which can dramatically affect a company's bottom line. New techniques in the field are evolving rapidly and expanded the application of ML to nearly limitless possibilities. Industries that depend on vast quantities of data—and need a system to analyse it efficiently and accurately, have embraced ML as the best way to build models, strategize, and plan.

3.2 Machine Learning Approaches

Machine learning approaches are traditionally divided into three broad categories, depending on the nature of the "signal" or "feedback" available to the learning system:

- Supervised learning
- Unsupervised learning
- Reinforcement learning

3.2.1 Supervised learning

Supervised learning is one of the machine learning approaches through which models are trained using perfectly labelled training data and on the basis of that models predicts the output. Types of supervised learning algorithms include active learning, classification and regression. Classification algorithms are used when the outputs are restricted to a limited set of values, and regression algorithms are used when the outputs may have any numerical value within a range. Similarity learning is an area of supervised machine learning closely related to regression and classification, but the goal is to learn from examples using a similarity function that measures how similar or related two objects are.

3.2.2 Unsupervised learning

Unsupervised learning algorithms take a set of data that contains only inputs, and find structure in the data, like grouping or clustering of data points. The algorithms, therefore, learn from test data that has not been labelled, classified or categorized. Instead of responding to feedback, unsupervised learning algorithms identify commonalities in the data and react based on the presence or absence of such commonalities in each new piece of data. A central application of unsupervised learning is in the field of density estimation in statistics, such as finding the probability density function. Though unsupervised learning encompasses other domains involving summarizing and explaining data features. 3.2.3 Semi-supervised learning

Semi-supervised learning falls between unsupervised learning (without any labelled training data) and supervised learning (with completely labelled training data). Some of the training examples are missing training labels, yet many machine-learning researchers have found that un-labelled data, when used in conjunction with a small amount of labelled data, can produce a considerable improvement in learning accuracy. In weakly supervised learning, the training

labels are noisy, limited, or imprecise; however, these labels are often cheaper to obtain, resulting in larger effective training sets.

3.2.4 Reinforcement learning

Reinforcement learning is feedback-based machine learning technique, and it aims to maximize the rewards by their hit and trial actions. In reinforcement learning the model learns automatically using feedbacks without any labeled data, unlike supervised learning and since there is no labelled data, the model is used to learn from its experiences only. Reinforcement learning is an area of machine learning concerned with how software agents ought to take actions in an environment so as to maximize some notion of cumulative reward.

3.3 Encryption

Encryption is a fundamental security mechanism used to protect data by converting readable information (plaintext) into an unreadable format (ciphertext) using mathematical algorithms and cryptographic keys. It ensures that only authorized parties with the correct decryption key can access the original data. In network communication, encryption algorithms such as Advanced Encryption Standard (AES) are commonly used to secure data transmission, while public-key algorithms like RSA help in securely exchanging encryption keys. Encryption plays a crucial role in maintaining confidentiality, integrity, and security of sensitive information in applications such as online banking, secure web browsing, email communication, and enterprise network systems.

3.4 Machine Learning Models

Performing machine learning involves creating a model, which is trained on some training data and then can process additional data to make predictions. Various types of models have been used and researched for machine learning systems.

3.4.1 Autoencoders and Dimensionality Reduction

An Autoencoder is a specialized neural network architecture designed for unsupervised feature learning through a process of data compression and reconstruction. Unlike supervised models that require labeled attack data, the Autoencoder learns to profile "normal" behaviour by mapping input features to a lower-dimensional bottleneck.

1. Encoding Process: The encoder transforms the high-dimensional input traffic feature vector $x \in \mathbb{R}n$ into a lower-dimensional latent representation z. Mathematically, this is expressed as:

$$z = f_{\phi}(x) = \sigma(Wx + b)$$

where $f\phi$represents the encoding function parameterized by weights $W$and biases b ,and σ is the activation function. This forces the model to capture only the most significant structural patterns of normal traffic behaviour.

2. Decoding and Reconstruction

The decoder attempts to reconstruct the original input from the compressed latent representation z:

$$\hat{x} = g_{\theta}(z) = \sigma(W'z+b')$$

where $g_{\theta}$is the decoding function parameterized by $\theta = \{W', b'\}$, and $\hat{x}$ denotes the reconstructed output.

3. Detection Hypothesis

The model is trained to minimize the Mean Squared Error (MSE), which serves as the reconstruction loss:

$$L(x, \hat{x}) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2$$

Theoretically, when the model is trained only on benign data, it becomes highly efficient at reconstructing normal traffic, resulting in a low reconstruction loss L. However, malicious traffic—such as botnet command-and-control (C2) or data exfiltration—exhibits different structural patterns that the model cannot accurately compress, leading to a high reconstruction error. Hence, the anomaly score for the Autoencoder is defined as:

$$S_{AE} = L(x, \hat{x})$$

3.4.2 Isolation Forest and the Isolation Principle

The Isolation Forest (iForest) shifts the detection paradigm from profiling normal data to explicitly isolating anomalies. This method is based on the assumption that anomalous instances are "few and different," making them statistically easier to separate.

1. Recursive Partitioning

The algorithm constructs an ensemble of random binary trees (iTrees). At each node, a feature is randomly selected along with a random split value between the minimum and maximum values of that feature. This recursive partitioning continues until each data instance is isolated.

2. Path Length Theory

For any data instance x, the path length $h(x)$is defined as the number of edges traversed from the root node to the terminating leaf node. Since anomalies lie in sparse regions of the feature space, they typically require fewer partitions to be isolated.

The normalized anomaly score is computed as:

$$s(x, n) = 2^{-\frac{\mathbb{E}[h(x)]}{c(n)}}$$

where $\mathbb{E}[h(x)]$is the average path length of instance $x$across all isolation trees, and $c(n)$is the average path length of an unsuccessful search in a Binary Search Tree, given by:

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}$$

Here, $H(n-1)$denotes the $(n-1)$-th harmonic number.

A score close to 1indicates a high likelihood of anomaly, whereas a score close to 0 indicates normal behavior. This property makes Isolation Forest robust to high-dimensional noise and highly efficient for real-time encrypted traffic monitoring.

3.5 Confusion matrix and its metrics

3.5.1 Confusion matrix

A confusion matrix is a performance evaluation tool used in classification problems to measure how well a model predicts different classes. It compares the actual (true) labels with the predicted labels and summarizes the results in a tabular form. The matrix helps identify correct predictions as well as different types of errors. It is especially useful for binary classification but can also be extended to multi-class problems. From the confusion matrix, important metrics such as accuracy, precision, recall, and F1-score can be calculated.

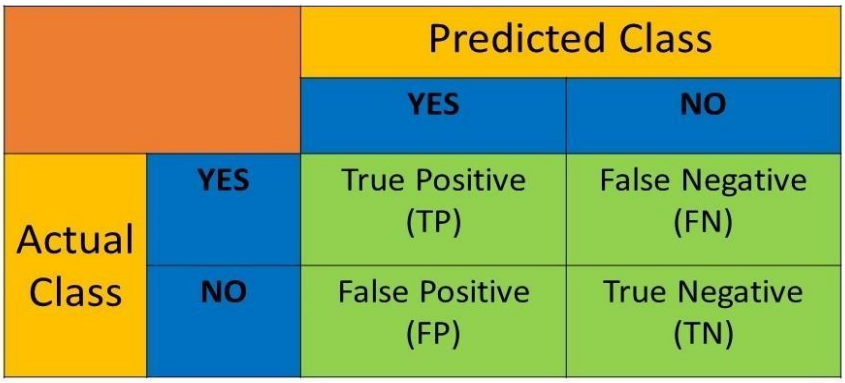


Figure 3.5: Confusion matrix with 4 parameters

- True Positive (TP): Model correctly predicts the positive class.
- True Negative (TN): Model correctly predicts the negative class.
- False Positive (FP): Model incorrectly predicts positive when it is actually negative
- False Negative (FN): Model incorrectly predicts negative when it is actually positive

3.5.2 Metrics

Accuracy

Accuracy is the most intuitive performance measure and it is simply a ratio of correctly predicted observation to the total observations.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

Precision is the ratio of correctly predicted positive observations to the total predicted positive observations.

$$Precision = \frac{TP}{TP + FP}$$

Recall (Sensitivity)

Recall is the ratio of correctly predicted positive observations to the all observations in actual class - yes.

$$Recall = \frac{TP}{TP + FN}$$

F1 Score

F1 Score is the weighted average of Precision and Recall. Therefore, this score takes both false positives and false negatives into account. Intuitively it is not as easy to understand as accuracy, but F1 is usually more useful than accuracy, especially if you have an uneven class distribution.

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

CHAPTER 4

REQUIREMENT SPECIFICATIONS

4.1 User Requirements

1. The user must be able to upload a CSV file containing network traffic data.
2. The system should automatically preprocess the uploaded dataset.
3. The trained hybrid model (Autoencoder + Isolation Forest) should analyze the data.
4. The system should detect different types of attacks such as:
    - XMRIGCC (CryptoMiner)
    - Brute Force
    - DoS
    - Benign Traffic
5. The system should calculate the percentage of each attack type.
6. The system should display results in:
    - Percentage format (e.g., XMRIGCC – 45%)
    - Bar graph visualization
7. The system should be easy to use with a simple web interface.
8. The system should provide fast detection results.

4.2 Software Requirements

Programming Languages:

- Python 3.x
- HTML5
- CSS
- JavaScript

Frameworks & Libraries:

- Flask (for backend web application)
- TensorFlow / Keras (for Autoencoder model)
- Scikit-learn (for Isolation Forest)
- Pandas (for CSV handling)
- NumPy (for numerical computation)

- Matplotlib / Chart.js (for bar graph visualization) Development Tools:
- VS Code / PyCharm
- Anaconda / Virtual Environment
- Web Browser (Chrome, Edge, etc.)

4.3 Hardware Requirements

The system was implemented and evaluated using a standard computing environment to ensure practical deploy ability. The hardware specifications are as follows:

Processor: Intel Core i5 multi-core processor

Memory: 8 GB RAM

Storage: 256 GB SSD

The experiments were conducted without requiring high-end computational infrastructure, demonstrating the scalability of the proposed approach for enterprise-level deployment.

CHAPTER 5

DESIGN AND IMPLEMENTATION

5.1 Methodology

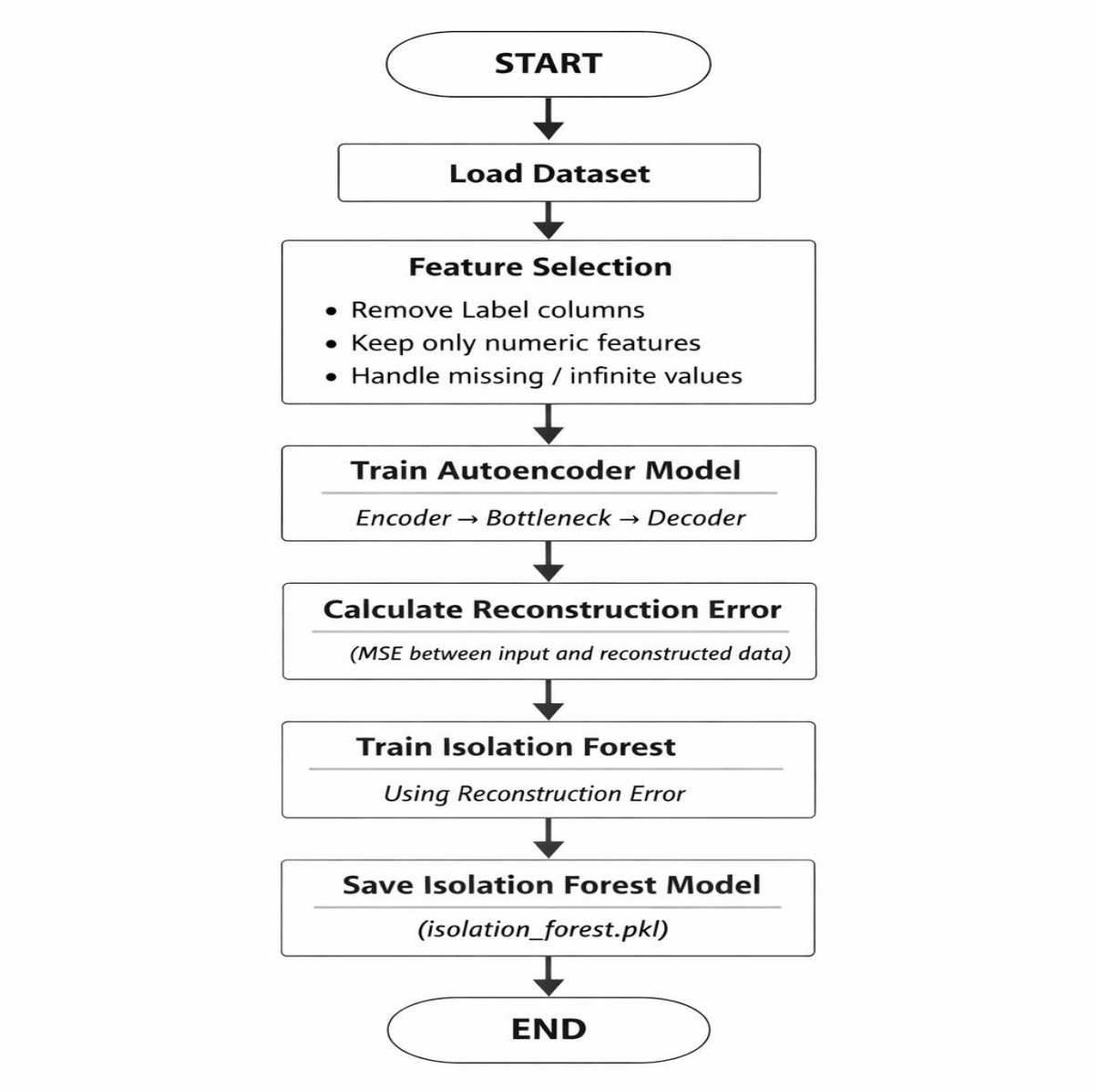


Figure 5.1 : System Flow

5.1.1 Algorithms Used:

1. Autoencoder

- Autoencoder is a deep learning neural network used for unsupervised learning and anomaly detection.
- It consists of two main parts: Encoder (compresses input data into a lower-dimensional representation) and Decoder (reconstructs the original data from the compressed form).
- The model is trained using normal network traffic data, so it learns the patterns and behaviour of legitimate traffic.
- When abnormal or malicious traffic is given as input, the model produces a high reconstruction error, indicating a potential anomaly.
- In this project, the reconstruction error from the Autoencoder is used as input for the Isolation Forest model to improve the accuracy of cyberattack detection.

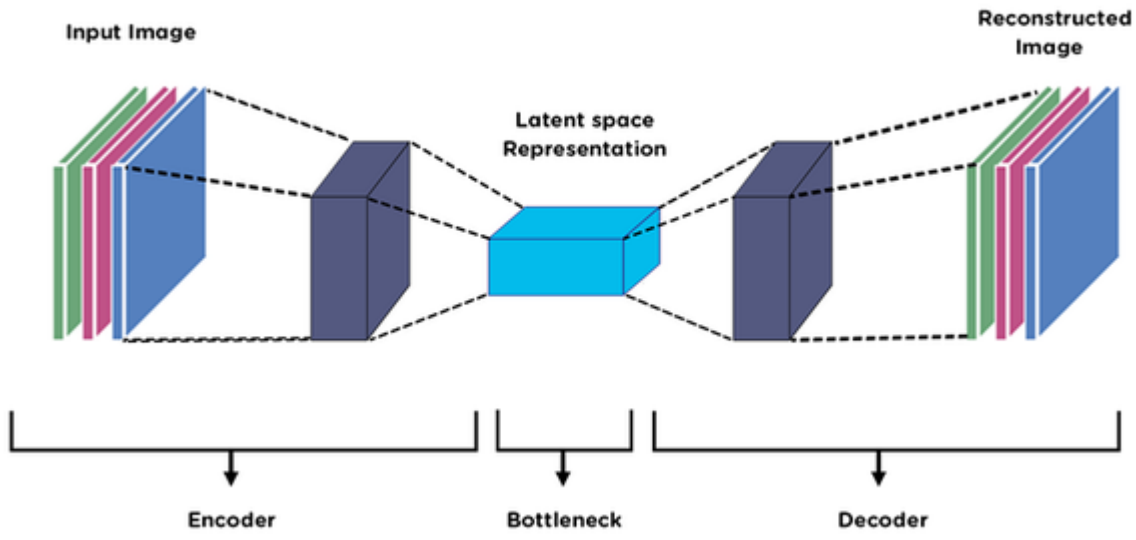


Figure 5.2 Autoencoder Architecture

2. Isolation Forest

- It works by randomly selecting features and splitting values to isolate observations in a tree structure called an Isolation Tree.
- Anomalies are isolated faster than normal data points because they are rare and different from the majority of the data.
- In this project, Isolation Forest analyzes the reconstruction error output from the Autoencoder to detect suspicious network traffic patterns.

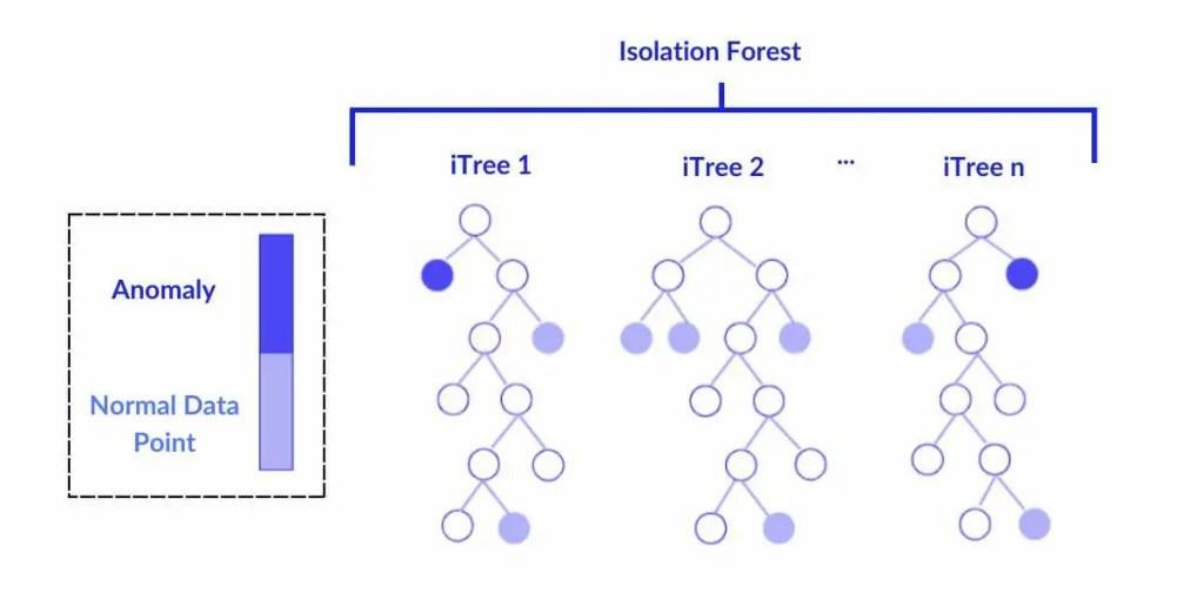


Figure 5.3 Isolation Forest Architecture

5.1.2 Data Input

The first stage of the proposed system involves data acquisition through a web-based interface. The user uploads a CSV file containing network traffic records. These records typically include various network flow features such as source and destination addresses, protocol information, packet statistics, flow duration, and other traffic-related attributes. Once uploaded, the system validates the file format and ensures that the data structure is suitable for further processing. This step enables seamless interaction between the user and the detection system.

5.1.3 Data Preprocessing

After successful upload, the dataset undergoes preprocessing to enhance data quality and improve model performance. Initially, missing or null values are identified and handled appropriately to prevent inconsistencies during model training and prediction. Categorical attributes are transformed into numerical form using encoding techniques to make them compatible with machine learning algorithms. Feature scaling is then applied through normalization or standardization methods to ensure that all input features contribute proportionally to the learning process. Additionally, irrelevant or redundant features are removed to reduce dimensionality and improve computational efficiency. This preprocessing stage ensures that the dataset is clean, consistent, and optimized for anomaly detection.

5.1.4 Autoencoder Model

The Autoencoder model is implemented as the first anomaly detection component in the hybrid framework. It is an unsupervised deep learning model trained primarily on normal (benign) network traffic data. The model consists of two main components: an encoder that compresses the input data into a lower-dimensional representation and a decoder that reconstructs the original input from the compressed representation.

During the prediction phase, the Autoencoder calculates the reconstruction error for each data instance. If the reconstruction error exceeds a predefined threshold, the instance is classified as anomalous. This approach enables the detection of unusual traffic patterns and previously unseen attacks, making it highly effective for zero-day intrusion detection.

5.1.5 Isolation Forest Model

The second component of the hybrid system is the Isolation Forest algorithm, an ensemble based anomaly detection technique. Unlike traditional models that profile normal data, Isolation Forest isolates anomalies by constructing multiple random decision trees. The algorithm randomly selects features and split values to partition the data.

Anomalous instances, being rare and significantly different from normal traffic, require fewer splits to be isolated in the tree structure. As a result, they obtain shorter path lengths compared to normal data points. This property allows Isolation Forest to efficiently identify malicious traffic in large-scale datasets with high accuracy and low computational cost.

5.1.6 Hybrid Detection Mechanism

To enhance detection performance, the outputs of the Autoencoder and Isolation Forest models are combined in a hybrid framework. This integration improves reliability by minimizing false positives and false negatives. When both models detect a data instance as anomalous, it is strongly classified as malicious traffic.

Based on prediction patterns and feature characteristics, the detected traffic is categorized into specific classes such as:

- XMRIGCC (CryptoMiner)
- Brute Force Attacks
- DoS (Denial of Service) Attacks
- Benign Traffic

This classification enables detailed insight into the nature and distribution of network intrusions.

5.1.7 Result Generation and Visualization

In the final stage, the system aggregates prediction results and computes the percentage distribution of each traffic category. The number of instances belonging to each attack type is calculated and converted into percentage format (for example, XMRIGCC – 45%).

To improve interpretability, the results are presented both numerically and graphically. A bar graph visualization is generated to display the comparative distribution of attack types within the uploaded dataset. This graphical representation allows users to quickly understand the overall security status of the network traffic.

5.2 Modules

The proposed Encrypted Network Traffic Anomaly Detection System using Hybrid Autoencoder and Isolation Forest is divided into the following functional modules:

1. Web Application Initialization Module

Function:

- Initializes the web application using the Flask framework.
- Handles communication between the frontend and backend.
- Manages user requests and server responses.

Purpose:

- Provides a web-based platform for users to interact with the anomaly detection system.

2. Model Loading Module

Function:

- Loads the trained Autoencoder model.
- Loads the Isolation Forest model.
- Loads the feature scaler and feature column list used during training.

Purpose:

- Allows the system to reuse trained models for prediction without retraining.

3. File Upload and Input Module

Function:

- Accepts network traffic datasets from the user.
- Supports dataset upload in CSV format.
- Converts uploaded data into a structured format.

Purpose:

- Acts as the data input point of the system for analyzing new traffic data.

4. Feature Alignment and Preprocessing Module

Function:

- Aligns dataset columns with training feature columns.
- Handles missing values and infinite values.
- Ensures all features are in the correct format.

Purpose:

- Maintains data consistency and compatibility with trained models.

5. Autoencoder Reconstruction Module

Function:

- Processes scaled network traffic data using the trained Autoencoder model.
- Reconstructs input data from compressed representation.
- Computes reconstruction error.

Purpose:

- Detects abnormal traffic patterns based on reconstruction differences.

6. Isolation Forest Detection Module

Function:

- Uses reconstruction error values as input.
- Applies Isolation Forest algorithm to identify anomalies.
- Assigns anomaly scores to each data sample.

Purpose:

- Detects suspicious or malicious network traffic activities.

7. Attack Categorization Module

Function:

- Separates traffic into normal and attack categories.
- Identifies known and unknown attack types based on dataset labels.

Purpose:

- Provides detailed classification of detected threats.

8. Result Processing Module

Function:

- Calculates percentage of normal traffic and attack traffic.
- Organizes detection results in a structured format.

Purpose:

- Summarizes results for easy interpretation and analysis.

9. API Response Module

Function:

- Sends detection results from backend to frontend.
- Uses structured data format for communication.

Purpose:

- Allows the web interface to display results clearly to the user.

10. Application Execution Module

Function:

- Runs the system as a web application.
- Processes user requests and prediction tasks.

Purpose:

- Enables the system to operate as a real-time anomaly detection platform.

CHAPTER 6
DATA EXPLORATION AND ANALYSIS

A comprehensive experimental evaluation was conducted to assess the performance of the proposed Hybrid Intrusion Detection System across three benchmark datasets: ALLFLOWMETER_HIKARI2022, CSE-CIC-IDS2018, and UNSW_NB15. These datasets represent varying network traffic characteristics and levels of attack complexity, enabling a thorough validation of detection capability and robustness. The Autoencoder and Isolation Forest models were first evaluated independently to analyze their individual strengths and limitations in anomaly detection.

6.1 Performance on ALLFLOWMETER_HIKARI2022 Dataset

The ALLFLOWMETER_HIKARI2022 dataset contains encrypted and diversified traffic flows, making it suitable for evaluating model stability.

The Autoencoder model achieved an accuracy of 93.66%, demonstrating strong capability in learning the underlying structure of normal traffic. The recall value of 0.7212 indicates effective anomaly detection capability. However, the precision value of 0.4725 suggests the presence of false positives when relying solely on reconstruction error for classification.

The Isolation Forest model achieved an accuracy of 91.53%. Although computationally efficient, it recorded lower precision (0.2673) and recall (0.2574) compared to the Autoencoder, indicating difficulty in distinguishing closely clustered anomalies.

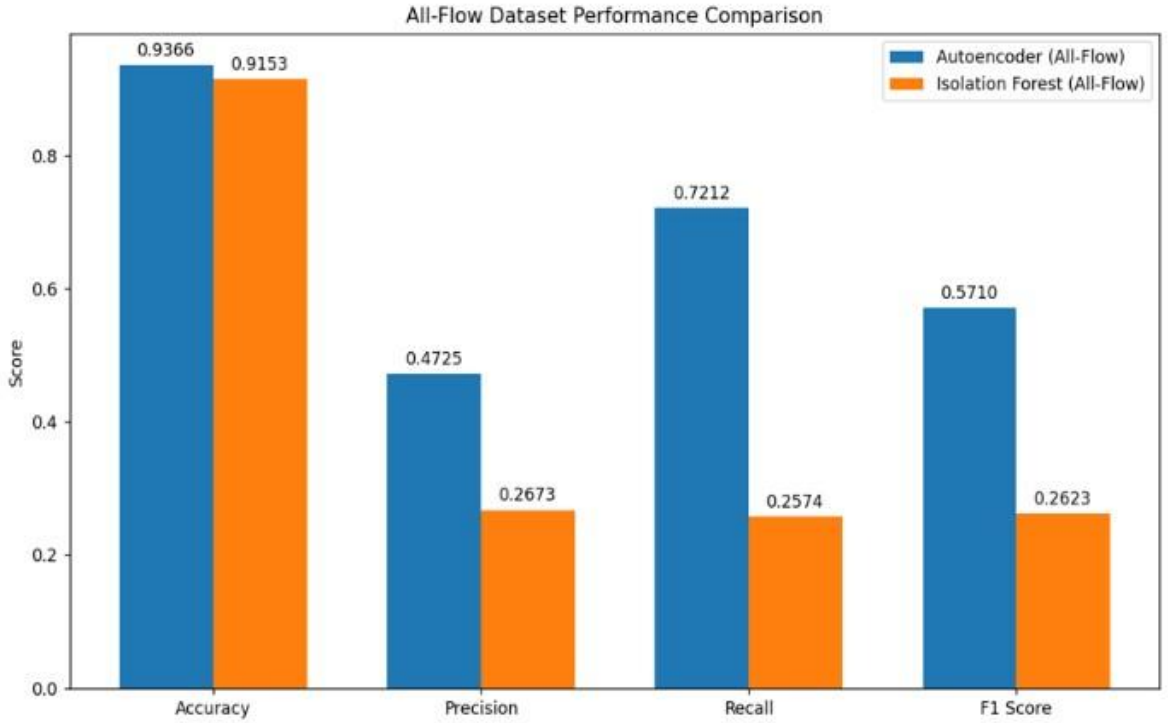


Figure 6.1: All flow meter performance

6.2. Performance on CSE-CIC-IDS2018 Dataset

The CSE-CIC-IDS2018 dataset was utilized for intermediate evaluation and rapid prototyping.

However, the recall value decreased compared to the previous dataset, indicating sensitivity to variations in traffic distribution.

The Isolation Forest achieved an accuracy of 86.83%, reflecting moderate detection performance. The relatively low precision value suggests an increased false-positive rate when used independently.

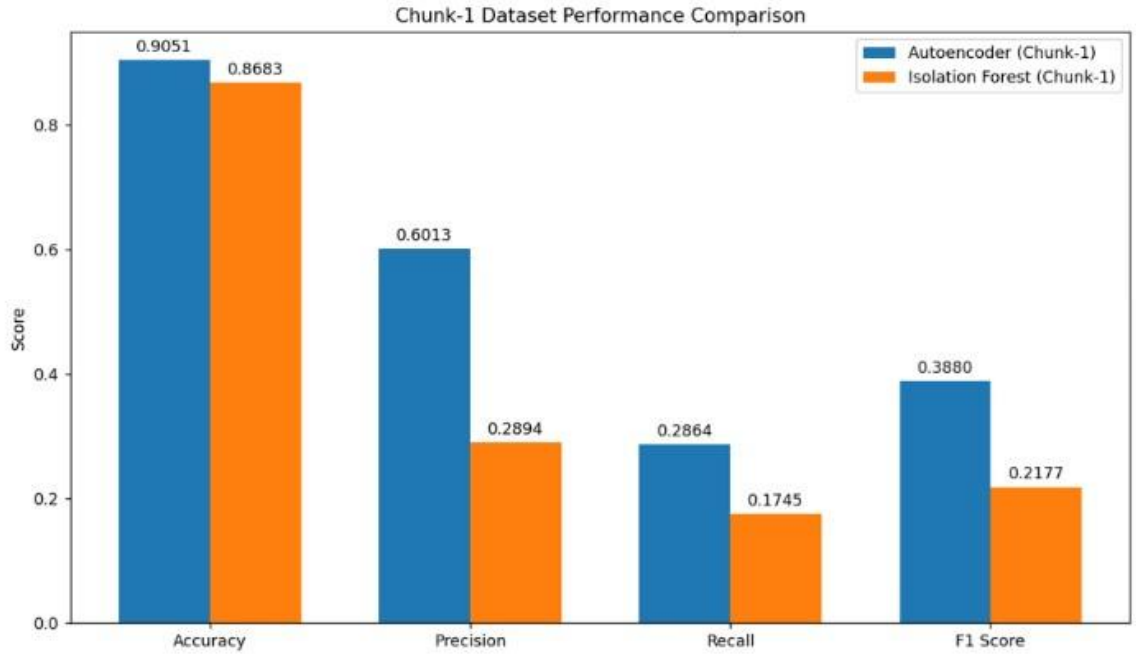


Figure 6.2: CSE-CIC-IDS2018 dataset performance

6.3 Performance on UNSW_NB15 Dataset

The UNSW_NB15 dataset is widely recognized as a benchmark dataset in intrusion detection research. It includes diverse and sophisticated attack patterns.

On this dataset, the Autoencoder exhibited reduced recall, indicating difficulty in reconstructing highly varied malicious traffic patterns. Although precision remained moderate, reconstruction-based detection alone proved insufficient for complex traffic distributions.

In contrast, the Isolation Forest demonstrated comparatively stronger recall and balanced precision. Its ability to isolate rare instances enabled it to detect anomalies not easily separable through reconstruction error analysis.

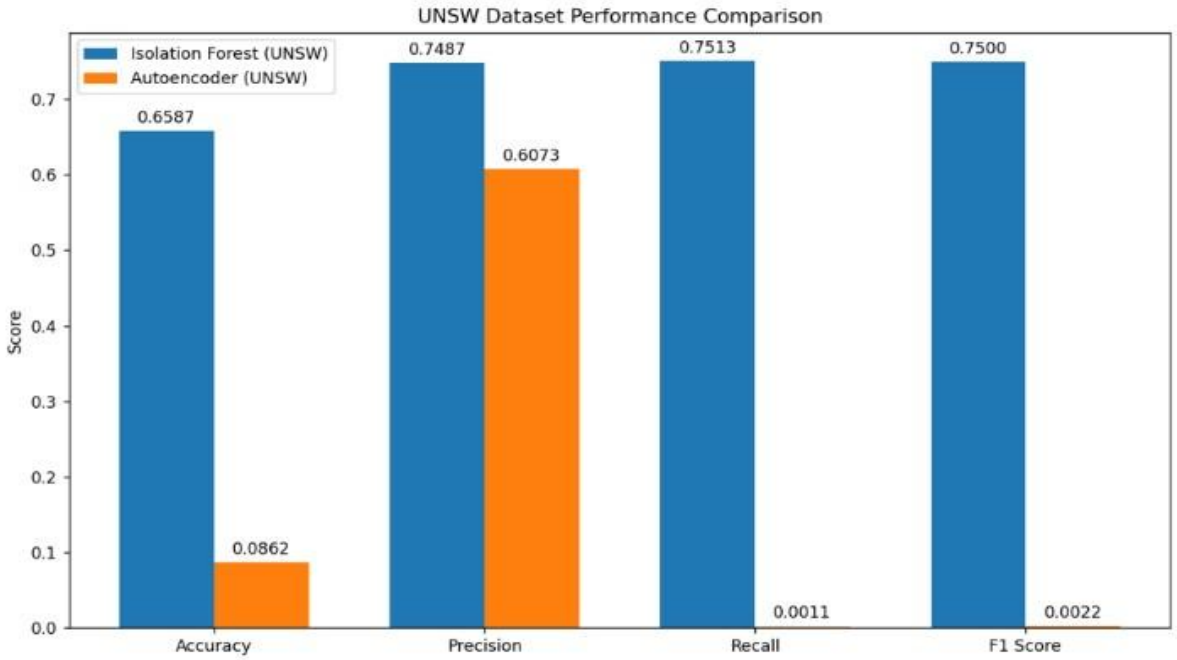


Figure 6.3: UNSW dataset performance

6.4. Comparative Analysis

Based on the comparative experimental evaluation across the three benchmark datasets, the ALLFLOWMETER_HIKARI2022 dataset was selected for final hybrid model development. This decision was made considering overall performance stability, balanced evaluation metrics, and consistent detection capability.

Dataset	**Model**	**Accuracy**	**Precision**	**Recall** **F1-Score**
ALLFLOWMETER_HIKARI2022	Autoencoder	0.9366	0.4725	0.7212 0.5710
	Isolation Forest	0.9153	0.2673	0.2574 0.2623
CSE-CIC-IDS2018	Autoencoder	0.9051	0.6013	0.2864 0.3880
	Isolation Forest	0.8683	0.2894	0.1745 0.2177
UNSW_NB15	Autoencoder	0.0862	0.6073	0.0011 0.0022
	Isolation Forest	0.6587	0.7487	0.7513 0.7500

Table 6.4.1 : Model Performance Results

On this dataset, the Autoencoder achieved a high accuracy of 93.66% with strong recall (0.7212), demonstrating effective learning of normal traffic patterns and reliable anomaly detection. The Isolation Forest also showed stable performance with an accuracy of 91.53%, contributing complementary anomaly isolation capability.

When the hybrid fusion mechanism was applied, the balance between precision and recall improved significantly, reducing false positives while maintaining satisfactory detection rates.

CHAPTER 7
MODELLING/IMPLENEMTATION

7.1 Overview of the Modelling

The primary objective of this phase is to design an intelligent and privacy-preserving framework capable of identifying malicious activities without inspecting encrypted packet contents. Traditional Intrusion Detection Systems rely heavily on Deep Packet Inspection, which becomes ineffective when network traffic is encrypted. Therefore, the proposed system focuses on behavioural analysis using traffic metadata such as flow duration, packet count, and protocol information. To achieve accurate detection, a Hybrid Machine Learning architecture is implemented by combining Deep Learning and Unsupervised Machine Learning techniques. The Autoencoder neural network learns normal traffic behaviour, while the Isolation Forest algorithm detects statistical deviations from learned patterns. This integration enhances anomaly detection performance while maintaining scalability and privacy compliance.

7.2 Data Preparation for Modelling

Before model training, the dataset undergoes systematic preprocessing to ensure data quality and consistency. Network traffic data collected from encrypted environments is first cleaned by removing duplicate records, missing values, and inconsistent entries.

Only meaningful metadata features related to network behaviour are retained for analysis. Features such as flow duration, packet length statistics, total packets, protocol type, and byte rate are used because they represent traffic behaviour even when packet contents are encrypted.

Non-numeric attributes are excluded to maintain compatibility with machine learning algorithms. After feature selection, feature normalization is applied using the StandardScaler technique to standardize numerical ranges and improve neural network convergence.

Furthermore, the original multi-class dataset is transformed into a binary classification structure, where:

- Normal traffic represents legitimate network behaviour
- Attack traffic represents anomalous or malicious behaviour

This transformation simplifies model learning and enhances generalization capability during anomaly detection.

7.3 Hybrid Machine Learning Model

The proposed system employs a hybrid anomaly detection architecture integrating an Autoencoder Deep Learning model and the Isolation Forest Unsupervised Machine Learning algorithm. The Autoencoder captures complex behavioural patterns of legitimate encrypted traffic, whereas the Isolation Forest identifies rare and abnormal observations. Combining both approaches overcomes limitations of single-model systems and significantly reduces false positives while improving detection reliability.

7.4 Autoencoder-Based Behaviour Learning

The Autoencoder is designed as a feed-forward neural network consisting of encoder and decoder components. The encoder compresses high-dimensional traffic features into a latent representation that captures essential characteristics of normal network behaviour. The decoder reconstructs the original input from this compressed representation.

Training is performed exclusively using benign traffic samples. During reconstruction, the model minimizes Mean Squared Error between input and output. Normal traffic produces low reconstruction error, while anomalous traffic generates higher error due to unfamiliar behavioural patterns. The reconstruction error therefore acts as the primary anomaly indicator used for further analysis.

7.5 Isolation Forest for Outlier Detection

Isolation Forest operates on the assumption that anomalous observations are rare and significantly different from normal traffic behaviour.

The algorithm constructs multiple random decision trees that recursively partition the dataset. Anomalies are isolated faster than normal samples and thus receive shorter path lengths within trees. Based on these path lengths, an anomaly score is assigned to each network flow, allowing effective detection of global irregularities.

7.6 Hybrid Decision Mechanism

Final anomaly classification is performed using a hybrid decision strategy that combines outputs from both models. A traffic instance is labelled as anomalous only when the Autoencoder reconstruction error exceeds a defined threshold and the Isolation Forest simultaneously identifies the instance as an outlier.

This dual-validation mechanism enhances robustness, minimizes false alarms, and improves overall system reliability in detecting zero-day and stealth attacks within encrypted environments.

7.7 Model Training Workflow

The complete model training process follows a structured pipeline including dataset loading and preprocessing, feature selection and normalization, Autoencoder training using benign traffic, reconstruction error computation, Isolation Forest training on anomaly scores, and model saving for deployment. The trained models and preprocessing objects are stored for reuse during real-time prediction and deployment.

1. Dataset loading and preprocessing
2. Feature selection and normalization using StandardScaler
3. Autoencoder training using benign traffic samples
4. Reconstruction error computation for all traffic instances
5. Isolation Forest training using reconstruction error values
6. Hybrid anomaly classification
7. Model saving for deployment

7.8 Model Deployment and Implementation

After training, the hybrid detection framework is deployed through a Flask-based web application.

The system provides an interactive interface where users can upload network traffic datasets in CSV format. Once the dataset is uploaded, the backend system automatically performs feature alignment, normalization, anomaly detection, and prediction using the trained models.

The system then generates prediction results that indicate whether the network traffic is normal or anomalous.To improve usability and monitoring, the detection results are displayed using visual dashboards that present:

- Traffic safety status
- Detected anomalies
- Statistical summaries of network flows

This implementation enables real-time monitoring of encrypted network environments while maintaining operational simplicity and user accessibility.

7.9 Performance Evaluation

The effectiveness of the proposed hybrid model is evaluated using standard classification metrics including Accuracy, Precision, Recall, and F1-Score. Experimental evaluation demonstrates improved detection capability compared to traditional signature-based systems and single machine learning models. The hybrid approach successfully balances high detection accuracy with low false-positive rates, making it suitable for practical cybersecurity deployment.

7.10 System Architecture

The system architecture describes the overall workflow of the proposed anomaly detection framework. It illustrates how encrypted network traffic data flows through different modules of the system, from data preprocessing to final anomaly detection and visualization. The architecture ensures efficient processing of traffic metadata without decrypting packet payloads, thereby preserving user privacy while maintaining strong security monitoring.

The main components of the architecture include:

1. Data Input Module

Network traffic datasets are collected and provided as input to the system.

2. Data Preprocessing Module

The dataset is cleaned by removing missing values, duplicates, and irrelevant attributes.

3. Feature Normalization Module

StandardScaler is applied to normalize feature values and ensure uniform data distribution.

4. Autoencoder Module

The Autoencoder learns normal traffic patterns and calculates reconstruction errors.

5. Isolation Forest Module

Isolation Forest analyses reconstruction errors and identifies abnormal network flows.

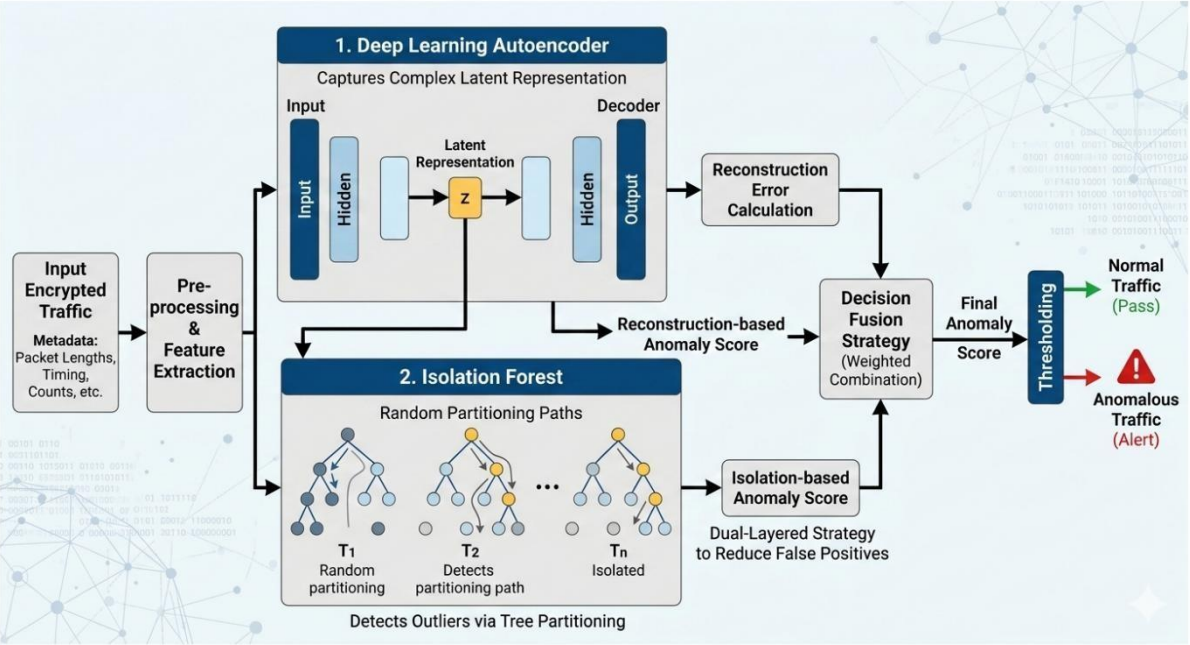
6. Hybrid Decision Engine

Outputs from both models are combined to determine the final anomaly classification.

7. Visualization Dashboard

Detection results are displayed through a user-friendly web interface for monitoring network security.

This architecture ensures scalability, privacy preservation, and efficient anomaly detection in encrypted network traffic environments.



7.10: System Architecture

CHAPTER 8
RESULTS AND CONCLUSION

8.1 Results

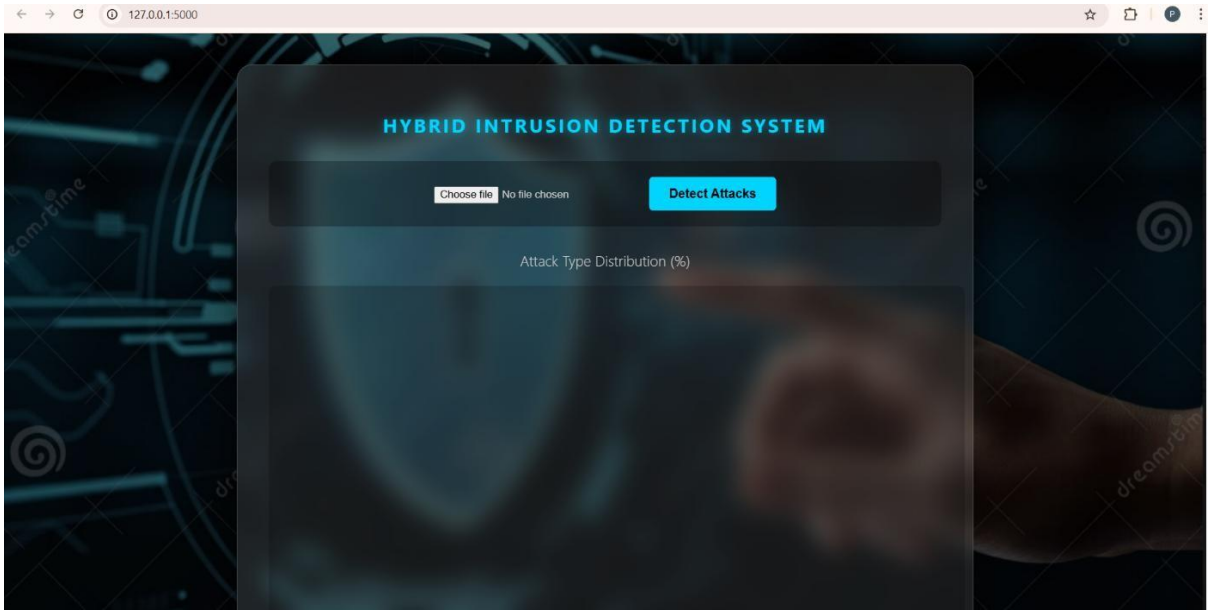


Figure 8.1.1: Web page for csv upload

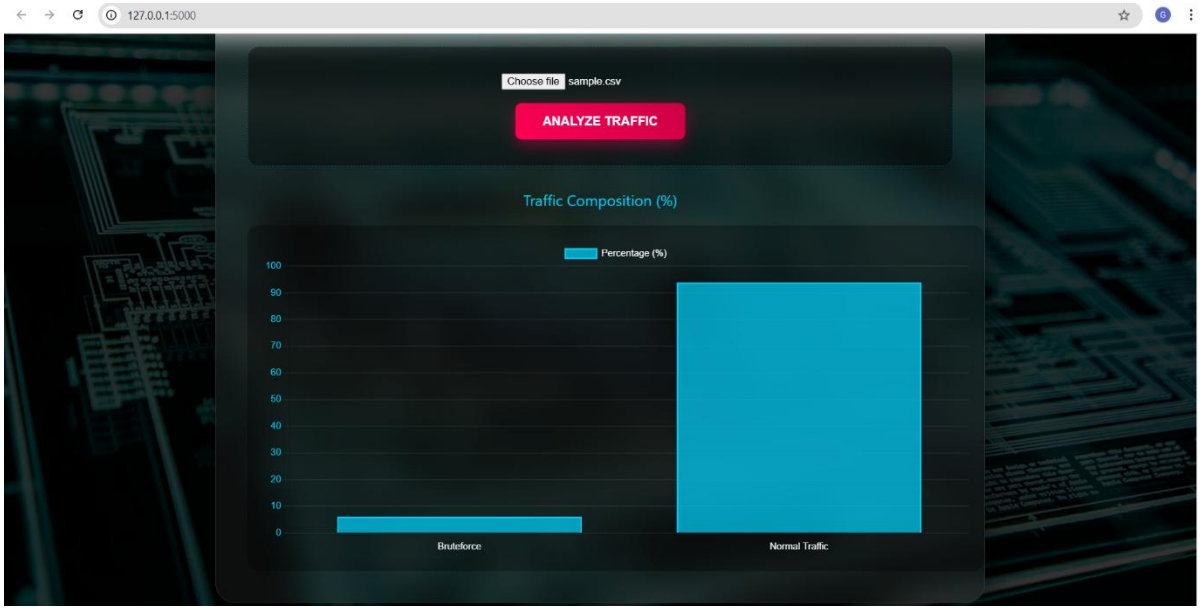


Figure 8.1.2: Detection of known attack

Figure 8.1.3: Detection of unknown attack

8.2 Conclusion and Future Work

Conclusion

This work presented a privacy-preserving anomaly detection framework designed to identify malicious activities in encrypted network traffic. As encryption becomes widely used in modern communication protocols, traditional intrusion detection systems that rely on payload inspection become less effective. To address this limitation, the proposed system analyzes only flow-level metadata without decrypting the traffic, ensuring both effective threat detection and preservation of user privacy.

The framework uses a hybrid modelling approach that combines an Autoencoder neural network and the Isolation Forest algorithm. The Autoencoder learns normal traffic behavior and detects deviations through reconstruction error, while the Isolation Forest identifies statistically rare patterns in network data. A hybrid decision mechanism integrates the outputs from both models to improve detection accuracy and reduce false positives.

Experimental evaluation using datasets such as ALLFLOWMETER_HIKARI2022, CSE-CIC-IDS2018, and UNSW_NB15 demonstrated that the hybrid model provides more stable and reliable anomaly detection across different traffic environments. The system was implemented using standard computational resources and deployed through a Flask-based web interface for monitoring detected anomalies. Overall, the proposed framework proves that analyzing encrypted traffic metadata can effectively detect cyber threats while maintaining privacy and regulatory compliance.

Future Work:

1. Hybrid Model Enhancement: Exploring advanced hybrid architectures by integrating additional machine learning or deep learning models such as CNNs, Graph Neural Networks, or Transformer-based models to improve feature extraction and anomaly detection performance.
2. Real-Time Traffic Monitoring: Integrating the framework with live packet capture tools to enable real-time monitoring and detection of anomalies in operational network environments.
3. Attack Classification Module: Extending the system to not only detect anomalies but also classify different types of cyberattacks such as DDoS, brute-force attacks, botnets, and infiltration attempts for better threat analysis.
4. Scalable Deployment: Optimizing the system for large-scale deployment in high-throughput enterprise or cloud networks using distributed computing and stream-processing technologies.

This project provides a strong foundation for future research in privacy-preserving network security systems, enabling intelligent monitoring solutions that can detect cyber threats effectively while maintaining the confidentiality of encrypted communications.

REFERENCES

[1] Y. Xu, J. Zhang, and K. Li, "Deep Learning-based Intrusion Detection Systems: A Survey," *arXiv preprint arXiv:2504.07839*, 2025.

[2] J. E. De Albuquerque Filho, et al., "A review of neural networks for anomaly detection," *IEEE Access*, vol. 10, pp. 112342–112367, 2022.

[3] Z. Zhang, et al., "Encrypted traffic classification via deep learning: A survey of the state of the art," *Computers, Materials & Continua*, vol. 71, no. 3, pp. 3345–3367, 2023.

[4] Y. Wang, et al., "Anomaly Detection in Encrypted Network Traffic Based on Machine Learning," *International Journal of Computer Networks*, 2022.

[5] T. Shapira, et al., "Encrypted Traffic Classification Using Machine Learning and Deep Learning," *IEEE Communications Magazine*, 2021.

[6] Ahmad, S., Shabbir, A. & Iqbal, S. Network intrusion detection system: A systematic study of machine learning and deep learning approaches. Secur. Priv. 4(3), e148. https://doi.org/10.1002/spy2.148 (2021).

[7] M. Lotfollahi, et al., "Deep Packet: A Novel Approach for Encrypted Traffic Classification Using Deep Learning," *IEEE Access*, vol. 8, 2020.

[8] García-Teodoro, P., Díaz-Verdejo, J., Sanz, S. & Camacho, D. Anomaly-based network intrusion detection: Techniques, systems and challenges. Comput. Netw. 53, 17–56. https://doi.org/10.1016/j.comnet.2019.107058 (2020).

[9] Zhang, F. & Xu, Y. Enhanced anomaly detection in network traffic using self-supervised pretraining. IEEE Trans. Cloud Comput. 11(1), 134–146. https://doi.org/10.1109/TCC.2023.3140227 (2023).

[10] Raman, S. & Singh, A. Scalable anomaly detection in encrypted traffic using graph neural networks. IEEE Trans. Cybern. 52(4), 3256–3269. https://doi.org/10.1109/TCYB.2022.3140323 (2022).

Appendix: A- Packages/Libraries, Tools used & Working Process

Python Programming Language

Python is a high-level, interpreted programming language widely used for general -purpose programming, scientific computing, and advanced machine learning applications. It supports dynamic typing and automatic memory management, which simplifies development and improves execution efficiency. Python follows multiple programming paradigms including object-oriented, functional, and procedural programming, making it highly flexible for different types of software development.

Python provides a large and comprehensive standard library, making it suitable for data analysis, artificial intelligence, cybersecurity, and web development. It uses advanced memory management techniques such as reference counting and garbage collection for optimized resource utilization. Python also supports dynamic name binding, meaning variables and functions are bound during execution, which enhances flexibility during runtime.

Python enables seamless integration with machine learning libraries, deep learning frameworks, data preprocessing tools, backend APIs, and frontend web applications. Its simplicity and readability make it ideal for implementing models such as Autoencoder and Isolation Forest for detecting network attacks from CSV datasets.

A.1 Libraries Used

NumPy

NumPy (Numerical Python) is an open-source Python library used for high-performance numerical and scientific computing. It provides a powerful N-dimensional array object called ndarray, which is faster and more memory-efficient than standard Python lists because it is implemented in optimized C code. NumPy supports various mathematical operations such as linear algebra, statistical functions, matrix multiplications, reshaping, broadcasting, slicing, and indexing.

NumPy forms the foundation of many data science and machine learning libraries such as Pandas, Scikit-learn, and TensorFlow. It enables efficient handling of large datasets and vectorized computations, which are essential for machine learning model training.

In this project, NumPy is used for:

- Handling network traffic feature arrays
- Performing mathematical computations
- Calculating reconstruction errors in Autoencoder
- Supporting tensor operations in deep learning models

NumPy improves computational efficiency when working with large network traffic datasets.

Import Syntax:

import numpy as np

Pandas

Pandas is an open-source data analysis and manipulation library used for handling structured data such as CSV files and tabular datasets. It provides powerful and flexible data structures such as Series (one-dimensional data) and DataFrame (two-dimensional tabular data) that make it easy to handle structured datasets like CSV files, Excel sheets, and database tables. Pandas is widely used in data science and machine learning because it allows users to clean data, filter rows and columns, handle missing values, perform statistical analysis, and prepare datasets for modeling. In simple terms, Pandas helps organize and analyze large amounts of data efficiently using Python.

In this project, Pandas is used for:

- Reading network traffic datasets (CSV files)
- Managing attack and benign labels
- Performing data preprocessing and cleaning
- Handling feature selection and dataset preparation

Pandas provides data structures such as Series and DataFrame which help in organizing dataset labels and metadata. Import Syntax:

import pandas as pd

Matplotlib

Matplotlib is a library used for plotting in the Python programming language and it is a numerical mathematical extension of NumPy. Matplotlib is most commonly used for visualization and data exploration in a way that statistics can be known clearly using different visual structures by creating the basic graphs like bar plots, scatter plots, histograms etc. Matplotlib is a foundation for every visualizing library and the library also offers a great flexibility with regards to formatting and styling plots. We can choose freely certain assumptions like ways to display labels, grids, legends etc.

In this project, Matplotlib is used for:

- Plotting accuracy curves
- Visualizing training and validation loss
- Displaying confusion matrix
- Analyzing dataset distribution

It helps in understanding model performance visually.

Import Syntax:

import matplotlib.pyplot as plt

Scikit-learn (Sklearn)

Scikit-learn (Sklearn) is the most useful and robust library for machine learning in Python. Scikit learn is an efficient, and beginner, user friendly tool for predictive data analysis and it provides a selection of tools for machine learning and statistical modeling including classification, regression, clustering and dimensionality reduction via a consistent interface in Python. This library is built upon different libraries such as NumPy, SciPy and Matplotlib. Scikit learn is used when identifying to which category an object likely belongs, predicting continuous values and grouping of similar objects into clusters.

In this project, Sklearn is used for:

- Splitting dataset into training and testing sets (train_test_split)
- Label encoding (LabelEncoder)
- Performance evaluation (accuracy, precision, recall, F1-score)

- Generating confusion matrix

Sklearn helps evaluate the classification performance of LSTM and Wav2Vec2 models.

Example Imports:

```
from sklearn.model_selection import
train_test_split from sklearn.preprocessing
import LabelEncoder
from sklearn.metrics import confusion_matrix, classification_report
```

TensorFlow / Keras

TensorFlow and Keras are deep learning frameworks used for building neural networks.

TensorFlow is an open-source deep learning framework developed by Google for building and deploying machine learning models. It supports numerical computation using data flow graphs and can efficiently run on CPUs, GPUs, and TPUs.

Keras is a high-level API integrated into TensorFlow that simplifies the process of building neural networks. It provides an intuitive interface for designing, training, and evaluating deep learning models.

In this project, TensorFlow/Keras is used to:

- Build the Autoencoder model
- Train the deep learning anomaly detection model
- Compile and optimize the neural network using Adam optimizer
- Prevent overfitting using EarlyStopping
- Evaluate reconstruction error for anomaly detection

The Autoencoder learns patterns of normal network traffic and detects anomalies based on reconstruction error. Combined with Isolation Forest, it forms a Hybrid Intrusion Detection System that improves attack detection accuracy.

Datasets Used

The model was trained using publicly available emotional speech datasets:

- CSE-CIC-IDS2018
- UNSW_NB15
- ALLFLOWMETER_HIKARI2022

A.2 Tools Used

Command Prompt (CMD)

Command Prompt (CMD) is a command-line interpreter application available in the Windows operating system. It is used to execute commands entered by the user to perform various system-level operations such as running programs, managing files, installing software packages, and executing scripts.

Unlike graphical user interfaces (GUI), Command Prompt works through text-based commands. It provides direct interaction with the system and allows developers to control and manage applications efficiently.

In this project, Command Prompt played an important role in the development and execution of the Vocal-Emotion Visualizer system.

Role of Command Prompt in This Project

The following operations were performed using Command Prompt:

1. Installing Required Libraries

All necessary Python packages such as NumPy, Pandas, TensorFlow, and others were installed using pip commands.

Example:

pip install numpy

pip install pandas

pip install flask

pip install keras

pip install sklearn

2. Creating and Activating Virtual Environment

To manage project dependencies separately, a virtual environment was created using CMD.

Example:

python -m venv emotion_env t_env\Scripts\activate
This ensures that project-specific libraries do not conflict with global Python installations.

3. Running Python Scripts

The deep learning models and backend applications were executed using CMD.

Example:

python
train_model.py
python app.py

4. Running Backend Server

For API integration using FastAPI or Flask, the backend server was launched through CMD.

Example:

python app.py

Advantages of Using Command Prompt

- Provides direct system-level control
- Faster execution compared to GUI tools
- Essential for backend development
- Required for running AI models and APIs
- Helps manage environments and dependencies

In this project, Command Prompt is used for:

- Installing required Python libraries using pip
- Creating virtual environments
- Running the backend server (FastAPI)
- Executing Python scripts
- Running model training programs
- Managing project dependencies

A.3 Working Process

A.3.1 Overview

The proposed Hybrid Intrusion Detection System (IDS) follows a structured pipeline that integrates data preprocessing, anomaly detection, deep learning–based reconstruction modeling, and attack percentage visualization. The system is designed to analyze uploaded network traffic CSV files and detect the percentage of different attack types using a hybrid modeling approach combining Autoencoder and Isolation Forest.

The complete working process consists of multiple stages including data acquisition, preprocessing, feature scaling, anomaly detection, attack classification, percentage calculation, and graphical visualization.

A.3.2 Input Acquisition

The system supports CSV-based input through a web interface.

1. CSV File Upload

The user uploads a network traffic dataset in .csv format through the web application. The file is transmitted to the backend server (Flask/FastAPI) for processing and analysis.

The uploaded dataset typically contains:

- Network flow features
- Traffic statistics
- Packet-related attributes
- Label column (if available for evaluation)

The backend API handles the uploaded file and forwards it to the preprocessing stage.

A.3.3 Data Preprocessing

After receiving the CSV file, preprocessing is performed to prepare the dataset for model inference.

The preprocessing steps include:

- Removing irrelevant or non-numeric columns
- Handling missing or null values
- Separating features and labels (if available)
- Converting categorical values into numeric form (if required)
- Feature selection (optional)

This step ensures the dataset is clean and suitable for machine learning models.

A.3.4 Feature Scaling

Before feeding the data into the models, feature scaling is applied using StandardScaler.

The scaling process includes:

- Standardizing features to zero mean and unit variance
- Transforming large-scale values into normalized range
- Ensuring uniform contribution of all features

Feature scaling improves model stability and prevents dominance of features with large numeric ranges.

A.3.5 Hybrid Model Architecture

The system follows a hybrid anomaly detection pipeline combining Autoencoder and Isolation Forest.

A.3.5.1 Autoencoder-Based Anomaly Detection

1. The preprocessed feature matrix is fed into the trained Autoencoder model.

2. The Autoencoder attempts to reconstruct normal network traffic patterns.

3. Reconstruction error is calculated as the difference between input and output.
4. A predefined threshold determines whether a sample is normal or anomalous.
5. High reconstruction error indicates potential attack traffic.

The Autoencoder captures complex nonlinear relationships in network data, improving anomaly detection capability.

A.3.5.2 Isolation Forest-Based Anomaly Detection

1. The scaled feature set is passed to the trained Isolation Forest model.
2. The algorithm isolates anomalies by randomly selecting features and split values.
3. Samples that require fewer splits to isolate are considered anomalies.
4. The model outputs binary predictions (Normal or Anomaly).

Isolation Forest enhances detection of rare and unknown attack patterns.

A.3.6 Attack Classification and Aggregation

After anomaly detection:

- The system identifies attack instances.
- If attack type labels are available, predictions are grouped by attack category such as:
    a. XMRIGCC
    b. Bruteforce
    c. DoS
    d. Benign
- The number of occurrences of each attack type is counted.
- The percentage of each attack type is calculated using:

Attack Percentage=Total instances/Number of attack instances×100

This provides a clear quantitative measure of network threat distribution.

A.3.7 Percentage Calculation

The system computes:

- Total number of records
- Total number of detected anomalies
- Individual attack type counts
- Percentage distribution for each attack category

Example Output:

- XMRIGCC – 45%
- Bruteforce – 30%
- DoS – 25%

This allows users to understand the severity and dominance of attack types in the dataset.

A.3.8 Visualization and Graph Generation

The calculated attack percentages are visualized using bar graphs.

The visualization stage includes:

- Generating bar charts for each attack type
- Displaying percentage values above bars
- Showing comparative distribution
- Dynamically updating results on the web interface

Matplotlib or Chart.js is used to render the graphical output.

This stage improves interpretability and makes results user-friendly.

A.3.9 Result Display and Response Handling

Finally, the system returns:

- Total records analyzed

- Number of detected anomalies
- Accuracy, Precision, Recall, and F1-score (if labels available)
- Attack percentage distribution
- Bar graph visualization

The frontend dynamically updates the interface using JavaScript to display results interactively after CSV upload.

Appendix: B

Sample Source Code with Execution

main.py(Hybrid Model training)

```
import numpy as np import pandas as pd import
joblib from sklearn.model_selection import
train_test_split from sklearn.preprocessing import
StandardScaler from sklearn.ensemble import
IsolationForest from tensorflow.keras.models
import Model from tensorflow.keras.layers import
Input, Dense from tensorflow.keras.callbacks
import EarlyStopping from
tensorflow.keras.optimizers import Adam df =
pd.read_csv("ALLFLOWMETER_HIKARI2022.c
sv")
# We use 'Label' for binary training logic (0=Benign, 1=Attack)
df["BinaryLabel"] = df["Label"].apply(lambda x: 0 if x == 0 else 1)
X = df.drop(columns=["Label", "BinaryLabel", "attack_category"], errors="ignore")

X = X.select_dtypes(include=[np.number])

X.replace([np.inf, -np.inf], np.nan,
inplace=True) X.fillna(0, inplace=True)
feature_columns = X.columns.tolist()
joblib.dump(feature_columns,
"feature_columns.pkl")
X_train, X_test, y_train, y_test = train_test_split(X, df["BinaryLabel"], test_size=0.3,
stratify=df["BinaryLabel"], random_state=42)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
joblib.dump(scaler, "scaler.pkl")
X_train_benign = X_train_scaled[y_train == 0]
input_dim = X_train_benign.shape[1]
```

```
input_layer = Input(shape=(input_dim,)) encoded =
Dense(64, activation="relu")(input_layer) encoded =
Dense(32, activation="relu")(encoded) decoded =
Dense(64, activation="relu")(encoded) output_layer =
Dense(input_dim, activation="linear")(decoded)


autoencoder = Model(input_layer, output_layer)
autoencoder.compile(optimizer=Adam(learning_rate=0.001),
loss="mse") autoencoder.fit(X_train_benign, X_train_benign,
epochs=30, batch_size=256, validation_split=0.1, verbose=1)
autoencoder.save("autoencoder_model.h5") train_recon_error =
np.mean(np.square(X_train_scaled -
autoencoder.predict(X_train_scaled)), axis=1) iso =
IsolationForest(n_estimators=300, contamination=0.06,
random_state=42) iso.fit(train_recon_error.reshape(-1, 1))
joblib.dump(iso, "isolation_forest.pkl") print("Training complete. All
models saved.")
```

Explanation of the Code

1. Dataset Loading

- The dataset ALLFLOWMETER_HIKARI2022.csv is loaded using Pandas.
- This dataset contains network flow features and attack labels.
- The data forms the foundation for training the Hybrid Intrusion Detection System.

2. Label Handling

- A new column named BinaryLabel is created.
- The original Label column is converted into binary format:
  - 0 → Benign
  - 1→ Attack (any non-zero value)

- This simplifies anomaly detection training.

This step prepares the dataset for binary classification logic used in anomaly detection.

3. Feature Selection and Cleaning

- Target columns (Label, BinaryLabel, attack_category) are removed from feature set.
- Only numeric columns are selected using select_dtypes().
- Infinite values are replaced with NaN.
- Missing values are filled with 0.

This ensures:

- The model receives only valid numeric input
- No missing or invalid values cause training errors

Additionally:

- Feature column names are saved using joblib.dump()
- This is crucial for maintaining consistency in the web application during prediction.

4. Train-Test Split and Feature Scaling

- The dataset is split into:
  - 70% Training data
  - 30% Testing data
- Stratified splitting ensures balanced distribution of benign and attack samples.

Feature scaling is performed using StandardScaler:

- Mean is centered to 0
- Standard deviation is scaled to 1

The trained scaler is saved as scaler.pkl for reuse during real-time prediction.

5. Autoencoder Training (Deep Learning Model)

The Autoencoder is trained only on Benign traffic data.

Steps involved:

- Extract only benign samples (y_train == 0)
- Define neural network architecture:
    - Input Layer
    - Dense Layer (64 neurons, ReLU)
    - Dense Layer (32 neurons, ReLU)
    - Dense Layer (64 neurons, ReLU)
    - Output Layer (same dimension as input)
- The model is compiled using:
    - Adam Optimizer
    - Mean Squared Error (MSE) loss
- The model is trained for:
    - 30 epochs
    - Batch size 256
    - 10% validation split

Purpose of Autoencoder:

- Learn normal network traffic patterns
- Reconstruct normal data accurately
- Produce high reconstruction error for attack data

The trained model is saved as:

autoencoder_model.h5

6. Reconstruction Error Calculation

- The trained Autoencoder predicts reconstructed output for training data.
- Reconstruction error is calculated using:

$$Reconstruction\ Error = Mean((Original - Reconstructed)^2)$$

- Higher error indicates abnormal behaviour.

This reconstruction error becomes the input for the next model.

7. Isolation Forest Training

- Isolation Forest is trained using reconstruction error values.
- Parameters used:
    - 300 trees (n_estimators=300)
    - Contamination rate = 0.06 (6% expected anomalies)
    - Random state = 42

Purpose of Isolation Forest:

- Detect anomalies based on isolation principle
- Identify rare attack patterns
- Improve detection accuracy when combined with Autoencoder

The trained model is saved as:

isolation_forest.pkl

8.Model Saving and Deployment Preparation

The following components are saved for deployment in the web application:

- feature_columns.pkl → Ensures correct feature order
- scaler.pkl → For scaling new uploaded data
- autoencoder_model.h5 → Deep learning anomaly detector
- isolation_forest.pkl → Final anomaly classifier

This enables the web application to:

- Load trained models
- Accept new CSV uploads
- Scale data correctly
- Detect attack percentage
- Display graphical results

B.2 Main Flask Application (Project Integration Layer)

import numpy as np import pandas as pd import joblib from
flask import Flask, render_template, request, jsonify from
tensorflow.keras.models import load_model app =
Flask(__name__) autoencoder =

```
load_model("autoencoder_model.h5", compile=False) scaler
= joblib.load("scaler.pkl") iso =
joblib.load("isolation_forest.pkl") feature_columns =
joblib.load("feature_columns.pkl") @app.route("/") def
home():
    return
render_template("index.html")
@app.route("/predict",
methods=["POST"]) def predict():
    file =
request.files["file"]
df = pd.read_csv(file)
    # 1. Feature Alignment (Ensure columns match training)
    X = df.reindex(columns=feature_columns, fill_value=0)
    X = X.replace([np.inf, -np.inf], np.nan).fillna(0)
    X_scaled = scaler.transform(X)     # 2. Hybrid Detection
reconstructed = autoencoder.predict(X_scaled, verbose=0)
recon_error = np.mean(np.square(X_scaled - reconstructed),
axis=1)
    # Use Isolation Forest decision function     scores =
iso.decision_function(recon_error.reshape(-1, 1))     threshold
= np.percentile(scores, 6) # Using your 6% threshold logic
df["Predicted_Attack"] = (scores < threshold).astype(int)
    # 3. Categorization (Mapping anomalies to attack names)
    # We look at 'attack_category' for the names     category_col =
"attack_category" if "attack_category" in df.columns else "Label"
detected = df[df["Predicted_Attack"] == 1]
    # Filter out benign/normal entries from the attack count     detected =
detected[~detected[category_col].astype(str).isin(['0', 'Benign', 'benign'])]     if
detected.empty:
```

```
        return jsonify({"Safe Traffic": 100})     # Count and
calculate percentages       counts =
detected[category_col].value_counts()     total = counts.sum()
percentages = {str(k): round((v/total)*100, 2) for k, v in
counts.items()}     return jsonify(percentages) if __name__ ==
"__main__":
    app.run(debug=True)
```

System Architecture Roles

This file acts as:

- Central Backend Controller
- Application Entry Point
- API Gateway Layer
- Hybrid Detection Engine